Modified Binary Search

As we know, whenever we are given a sorted Array or LinkedList or Matrix, and we are asked to find a certain element, the best algorithm we can use it the binary Search.

Order-agnostic Binary Search

Problem Statement #

Given a sorted array of numbers, find if a given number 'key' is present in the array. Though we know that the array is sorted, we don't know if it's sorted in ascending or descending order. You should assume that the array can have duplicates.

Write a function to return the index of the 'key' if it is present in the array, otherwise return -1.

Example 1:

```
Input: [4, 6, 10], key = 10
Output: 2
```

To make things simple, let's first solve this problem assuming that the input array is sorted in ascending Order. Here are the sets of steps for binary Search

1) let's assume start is pointing to the first index & end is pointing to the last index of input array (let's call it arr). This means:

```
int start = 0;
int end = arr.length - 1;
```

2) First, we will find the middle of start and end. An easy way to find middle would be: middle = (start + end) / 2. For Java and C++, this equation will work for most cases, but when start or end is large, this equation will give us the wrong result due to integer overflow. Imagine that start is equal to the max range of an integer (eg for java: int start = Integer.MAXVALUE Now adding anything to start will result in integer overflow. Since we need to add both the number first to evaluate our equation, an overflow might occur. The safest way to find the middle of two numbers without getting an overflow is as follows →

```
middle = start + (end - start) / 2
```

↳ not relevant for python as we don't have the integer overflow problem in pure python.

3) Next, we will see if the 'Key' is equal to the number at index middle. if it is equals we return middle as required index.

4) if 'Key' is not equal to number at index middle, we have to check two things

- if Key < arr[middle], then we can conclude that the key will be smaller than all the number after index middle as the array is sorted in the ascending order hence we can reduce our to end = mid - 1.

- if Key > arr[middle], then we can conclude that the key will be greater than all number before index middle as the array is sorted in the ascending order. hence, we can reduce our search to start = mid + 1.

5) We will repeat step 2-4 with new ranges of start to end. If at any time start become greater than end, this means that we can't find the 'Key' in the input array and we must return -1.

6) And have to the reverse if the array is desending order, we have to reverse the updating logic.

```
# include < bits/ stdc++. h >
using namespace std;

class BinarySearch {

    public:
        static int search( const vector<int>& arr, int key){
        int start = 0;
        int end = arr. size( ) - 1;
        bool isAscending = arr[start] < arr[end];
        while ( start <= end ){
            int mid = start + (end - start)/ 2;
            if( key == arr[mid] ){
                return mid;
            }
```

```
if (isAscending){ // ascending order
    if(key < arr[mid]) {
        end = mid - 1;
    } else {
        start = mid + 1;
    }
} else { // descending Order
    if (key > arr[mid]){
        end = mid - 1;
    } else {
        start = mid + 1;
    }
}
return -1;
}
};
```

Problem – Ceiling of a Number

Problem Statement #

Given an array of numbers sorted in an ascending order, find the ceiling of a given number 'key'. The ceiling of the 'key' will be the smallest element in the given array greater than or equal to the 'key'.

Write a function to return the index of the ceiling of the 'key'. If there isn't any ceiling return -1.

Example 1:

```
Input: [4, 6, 10], key = 6
Output: 1
Explanation: The smallest number greater than or equal to '6' is '6' ha
ving index '1'.
```

Example 2:

```
Input: [1, 3, 8, 10, 15], key = 12
Output: 4
Explanation: The smallest number greater than or equal to '12' is '1
5' having index '4'.
```

Example 3:

```
Input: [4, 6, 10], key = 17
Output: -1
Explanation: There is no number greater than or equal to '17' in the gi
ven array.
```

```cpp
#include <bits/stdc++.h>
using namespace std;

class CeilingOfANumber {
  public:
    static int searchCeilingOfANumber(const vector<int>& arr, int key){
        if(key > arr[arr.size()-1]){
            return -1; // if the 'key' is bigger than the biggest element
        }
        int start = 0;
        int end = arr.size()-1;
        while(start <= end){
            int mid = start + (end - start)/2;
            if(key < arr[mid]){
                end = mid - 1;
            }else if(key > arr[mid]){
                start = mid + 1;
            }else{ // found the key
                return mid;
            }
        }
        // since the loop is running until 'start <= end', so at the end of the while loop, 'start == end+1' we are not able to find the element in the given array, so the next big number will be arr[start].
        return start;
    }
};
```

Problem - Next Letter

Given an array of lowercase letters sorted in ascending order, find the **smallest letter** in the given array **greater than a given 'key'**.

Assume the given array is a **circular list**, which means that the last letter is assumed to be connected with the first letter. This also means that the smallest letter in the given array is greater than the last letter of the array and is also the first letter of the array.

Write a function to return the next letter of the given 'key'.

Example 1:

```
Input: ['a', 'c', 'f', 'h'], key = 'f'
Output: 'h'
Explanation: The smallest letter greater than 'f' i
s 'h' in the given array.
```

```
Input: ['a', 'c', 'f', 'h'], key = 'm'
Output: 'a'
Explanation: As the array is assumed to be circular, th
e smallest letter greater than 'm' is 'a'.
```

Example 4:

```
Input: ['a', 'c', 'f', 'h'], key = 'h'
Output: 'a'
Explanation: As the array is assumed to be circular, th
e smallest letter greater than 'h' is 'a'.
```

```cpp
#include <bits/stdc++.h>
using namespace std;

class NextLetter {
public:
    static char searchNextLetter(const vector<char>& letters, char key){
        int n = letters.size();
        if (key < letters[0] || key > letters[n-1]){
            return[0];
        }

        int start = 0;
        int end = n - 1;
        while (start <= end){
            int mid = start + (end - start) / 2;
            if ( key < letters[mid]){
                end = mid - 1;
            }else {  // if (key >= letter[mid])
                start = mid + 1;
            }
        }

    }
    // since the loop is running until 'start <= end', so at the end of the
       while loop, 'start == end + 1'
```

```
        return letters[start % n];
    }
};
```

Problem - Search in a Sorted Infinite Array

Given an infinite sorted array (or an array with unknown size), find if a given number 'key' is present in the array. Write a function to return the index of the 'key' if it is present in the array, otherwise return -1.

Since it is not possible to define an array with infinite (unknown) size, you will be provided with an interface `ArrayReader` to read elements of the array. `ArrayReader.get(index)` will return the number at index; if the array's size is smaller than the index, it will return `Integer.MAX_VALUE`.

Example 1:

```
Input: [4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30], key = 16
Output: 6
Explanation: The key is present at index '6' in the array.
```

```
Input: [4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30], key = 11
Output: -1
Explanation: The key is not present in the array.
```

Example 3:

```
Input: [1, 3, 8, 10, 15], key = 15
Output: 4
Explanation: The key is present at index '4' in the array.
```

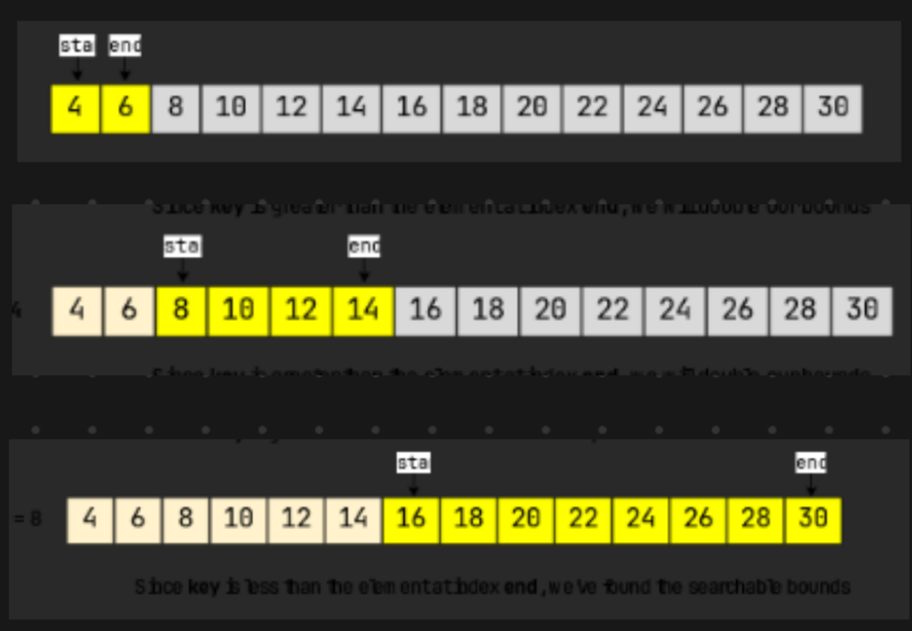
Example 4:

```
Input: [1, 3, 8, 10, 15], key = 200
Output: -1
Explanation: The key is not present in the array.
```

This problem follows the binary Search pattern. Since Binary Search helps us find a number in a sorted array efficiently, we can use a modified version of the binary Search to find the 'key' in an infinite sorted array.

The only issue with apply binary search in this problem is that we don't know the bound of the array. To handle this situation, we will first find the proper bounds of the array where we can perform a binary Search.

An efficient way to find the proper bound is to start at the beginning of the array with the bound's size as '1' and exponentially increase the bound size (ie double it) until we find the bounds that can have the key.



```
#include <bits/stdc++.h>
using namespace std;
class ArrayReader {
    public:
      vector<int> arr;
      ArrayReader(const vector<int> &arr) { this -> arr = arr; }
      virtual int get (int index){
        if (index >= arr.size()) {
            return numeric_limits<int>::max();
        }
        return arr[index];
      }
};
class SearchInfiniteSortedArray {
  public:
    static int search(ArrayReader *reader, int key){
        while( start<=end) {
            int mid = start + (end - start) / 2;
            if (key < reader->get(mid)) {
                end = mid - 1;
            }else if (key > reader->get(mid)) {
                start = mid+1;
            }else{
                return mid;
            }
        }
        return -1;
    }
};
```

Minimum Difference Element

Given an array of numbers sorted in ascending order, find the element in the array that has the minimum difference with the given 'key'.

Example 1:

```
Input: [4, 6, 10], key = 7
Output: 6
Explanation: The difference between the key '7' and '6' is minimum tha
n any other number in the array
```

Example 2:

```
Input: [4, 6, 10], key = 4
Output: 4
```

```cpp
#include <bits/stdc++.h>
using Namespace std;

class MinimumDifference{
public:
    static int searchMinDiffernceElement(const vector<int>& arr, int key){
        if (key < arr[0]) {
            return arr[0];
        }
        if(key > arr[arr.size() - 1]){
            return arr[arr.size()- 1];
        }
        int start = 0;
        int end = arr.size()-1;
        while( start <= end){
            int mid = start + (end - start )/2;
            if (key < arr[mid]) {
                end = mid + 1;
            } else if ( key > arr[mid]){
                start = mid -1;
            } else {
```

```
            return arr[mid];
        }
    }
    // at the end of the while loop, 'start == end + 1'
    // we are able to find the element in the given array
    // return the element which is closest to the 'key'
    if((arr[start] - key) < (key - arr[end])){
        return arr[start];
    }
    return arr[end];
    }
};
```

Problem - Number Range

range of a given number 'key'. The range of the 'key' will be the first and last position of the 'key' in the array.

Write a function to return the range of the 'key'. If the 'key' is not present return [-1, -1].

Example 1:

```
Input: [4, 6, 6, 6, 9], key = 6
Output: [1, 3]
```

Example 2:

```
Input: [1, 3, 8, 10, 15], key = 10
Output: [3, 3]
```

```
#include <bits/stdc++.h>
using namespace std;

class FindRange{
  public:
    static pair<int, int> findRange(const vector<int>& arr, int key){
        pair<int, int> result(-1, -1);
        result.first = search(arr, key, false);
        if(result.first != -1){ // No need to search, if 'key' is not present
                                in the input array.
            result.second = search(arr, key, true);
        }
        return result;
```

```
    }
private:
    static int search(const vector<int>& arr, int key, bool findMaxIndex){
        int KeyIndex = -1;
        int start = 0;
        int end = arr.size()-1;
        while (start <= end){
            int mid = start + (end-start)/2
            if( key < arr[mid]){
                end = mid -1;
            } else if ( key > arr[mid]){
                end = mid + 1;
            } else { // key = arr[mid]
                KeyIndex = mid;
                if( findMaxIndex) {
                    start = mid+1;
                }else {
                    end = mid -1;
                }
            }
        }
        return keyIndex;
    }
};
```

Problem - Bitonic Array Maximum

Find the maximum value in a given Bitonic array. An array is considered bitonic if it is monotonically increasing and then monotonically decreasing. Monotonically increasing or decreasing means that for any index `i` in the array `arr[i] != arr[i+1]`.

Example 1:

```
Input: [1, 3, 8, 12, 4, 2]
Output: 12
Explanation: The maximum number in the input bitonic array is '12'.
```

Example 2:

```
Input: [3, 8, 3, 1]
Output: 8
```

```cpp
#include<bits/stdc++.h>
using namespace std;
class MaxInBitonicArray{
public:
    static int findMax(const vector<int>& arr){
        int start=0;
        int end = arr.size()-1;
        while(start<end){  // as start==end represent maximum.
            int start=0;
            int end = arr.size()-1;
            if(arr[mid]>arr[mid+1]){
                end = mid;
            }else{
                start = mid+1;
            }
        }
        // at the end of the while loop, 'start==end'
        return arr[start];
    }
};
```

Problem - Search Bitonic Array

Search Bitonic Array (medium) #

Given a Bitonic array, find if a given 'key' is present in it. An array is considered bitonic if it is monotonically increasing and then monotonically decreasing. Monotonically increasing or decreasing means that for any index `i` in the array `arr[i] != arr[i+1]`.

Write a function to return the index of the 'key'. If the 'key' is not present, return -1.

Example 1:

```
Input: [1, 3, 8, 4, 3], key=4
Output: 3
```

```
Input: [3, 8, 3, 1], key=8
Output: 1
```

Example 3:

```
Input: [1, 3, 8, 12], key=12
Output: 3
```

Example 4:

```
Input: [10, 9, 8], key=10
Output: 0
```

1) First, we can find the index of the maximum value of the bitonic array, similar to bitonic Array Maximum. Let's call the index of the maximum number maxIndex.

2) Now, we can break the array into two sub-arrays:
   - Array from index '0' to maxIndex, sorted in ascending order.
   - Array from index maxIndex+1 to array_length-1, sorted in descending order.

3) We can then call binary Search separately in these two arrays to search the 'key'. We can use the same Order-agnostic Binary Search for searching.

```
#include <bits/stdc++.h>
using namespace std;
class SearchBitonicArray {
  public:
    static int search(const vector<int> &arr, int Key) {
        int maxIndex = findMax(arr);
        int KeyIndex = binarySearch(arr, key, 0, maxIndex);
        if (keyIndex!=-1){
            return KeyIndex;
        }
        return binarySearch(arr, key, maxIndex+1, arr.size()-1);
    }
    // find index of the maximum value in a bitonic array.
static int findMax(const vector<int> &arr){
    int start = 0;
    int end = arr.size()-1;
    while(start < end){
```

```
        int mid = start + (end - start) / 2;
        if(arr[mid] > arr[mid+1]){
            end = mid;
        }else{
            start = mid+1;
        }
    }
    // at the end of the whole loop, 'start == end'
    return start;
}

private:
    // order-agnostic binary search.
    static int binarySearch(const vector<int>& arr, int key, int start, int end){
        while(start <= end){
            int mid = start + (end - start) / 2;

            if(key == arr[mid]){
                return mid;
            }

            if(arr[start] < arr[end]){ // ascending order
                if(key < arr[mid]){
                    end = mid-1;
                }else{ // key > arr[mid]
                    start = mid+1;
                }
            }else{ // descending order
                if(key > arr[mid]){
```

```
            end = mid - 1;
        } else {
            start = mid + 1;
        }
    }
  }
  return -1;
 }
};
```

Problem → Search in Rotated Array

Given an array of numbers which is sorted in ascending order also rotated by some arbitrary number, find if a given 'key' is present in it.

Write a function to return the index of the 'key' in the rotated array. If the 'key' is not present, return -1. You can assume that the given array does not have any duplicates.

Example 1:

```
Input: [10, 15, 1, 3, 8], key = 15
Output: 1
Explanation: '15' is present in the array at index '1'.
```

Example 2:

```
Input: [4, 5, 7, 9, 10, -1, 2], key = 10
Output: 4
Explanation: '10' is present in the array at index '4'.
```


After calculation of the middle, we can compare the number at indices start and middle. This will give us two options:

1) if arr[start] <= arr[middle], the number from start to middle are sorted in ascending order

2) Else, the number from middle + 1 to end are sorted in ascending order.

Once we know which part of the array is sorted, it is easy to adjust our ranges. For example, if option - 1 is true, we have two choices:

1) By comparing the 'key' with the number at index start and middle we can easily find out if the 'key' lies between indices start and middle; if it does, we can skip the second part ⇒ end = middle - 1.

2) Else, we can skip the first part ⇒ start = middle + 1.

Since arr[start] <= arr[middle], we can conclude that all number from start -> middle are sorted in ascending order.

by looking at the sorted part, from its start and end, we can conclude that that the key can't be in the sorted part, hence start = middle + 1.

Since arr[start] > arr[middle], we can conclude that all numbers from middle → end are sorted in ascending order. By looking at the sorted part, from its start and end, we can conclude that the key can't be in sorted part hence end = middle - 1.


```
#include <bits/stdc++.h>
using namespace std;

class SearchRotatedArray {
 public:
   static int search(const vector<int>& arr, int key){
      int start = 0;
      int end = arr.size() - 1;
      while(start <= end) {

         int mid = start + (end - start)/2;
         if (arr[mid] == key){
             return mid;
         }
```

```
if(arr[start] <= arr[mid]){ // left side is sorted in ascending order
    if ( key >= arr[start] && key < arr[mid]){
        end= mid - 1;
    }else {
        if (key > arr[mid] && key <= arr[end]){
            start = mid+1;
        }else { // right side is sorted in a
            end= mid - 1;
        }
    }
}
// we are not able to find the element in the given array
return -1;
}
};
```

Problem - Rotation Count

Given an array of numbers which is sorted in ascending order and is rotated 'k' times around a pivot, find 'k'.

You can assume that the array does not have any duplicates.

Example 1:

```
Input: [10, 15, 1, 3, 8]
Output: 2
Explanation: The array has been rotated 2 times.
```

Original array: 1 3 8 10 15

Array after 2 rotations: 10 15 1 3 8

```
Input: [4, 5, 7, 9, 10, -1, 2]
Output: 5
Explanation: The array has been rotated 5 times.
```

Original array: -1 2 4 5 7 9 10

Array after 5 rotations: 4 5 7 9 10 -1 2

Example 3:

```
Input: [1, 3, 8, 10]
Output: 0
Explanation: The array has not been rotated.
```

```
#include <bits/stdc++.h>
using namespace std;

class RotationCountOfRotatedArray {
  public:
    static int countRotations(const vector<int> &arr){
        int start = 0;
        int end = arr.size() - 1;
```

```
while ( start < end){
    int mid = start + (end - start )/ 2;

    if ( mid < end && arr[mid] > arr[mid + 1]){

    // if mid is greater than the next element.
        return mid + 1;
    }
    if ( mid > start && arr[mid-1] >arr[mid]) {

    // if mid is smaller than the previous element
        return mid;
    }

    if ( arr[start] < arr[ mid]) { // left side is sorted, so the pivot is on
                                      right side.
        start = mid + 1;
    }else { // right side is sorted . so the pivot is on the left side.
        end = mid - 1;
    }
  }
  return 0; // the array has not been rotated.
}
};
```